

Smart Data Processing & Validation Dashboard

Project Report

Supriya Ojha

1. Introduction

This report documents the design, development, and implementation of a Python-based web application that provides interactive data analytics capabilities. The project leverages Flask as the backend framework and integrates several advanced Python programming concepts including multithreading, multiprocessing, object-oriented programming, closures, generators, iterators, decorators, and mixins.

The application allows users to register, upload CSV or JSON datasets, and receive statistical insights with dynamically generated visualisations — all served through a browser-based interface.

2. Project Objectives

- Build a full-stack web application using Python (Flask) and HTML/CSS.
- Demonstrate advanced Python concepts: OOP, decorators, generators, closures, iterators, mixins, and abstract classes.
- Implement concurrent processing via Python multithreading and multiprocessing.
- Perform real-time statistical analysis (mean, median, standard deviation) on user-uploaded datasets.
- Auto-generate charts (line, bar, histogram) using Matplotlib and render them on the dashboard.
- Apply input validation using regular expressions.
- Persist results using JSON serialisation.

3. Project Structure

The project uses a clean backend/frontend separation. Note: While the assignment specified a flat `smart_data_system/` structure, the project adopts a `backend/` and `frontend/` split for better separation of concerns and maintainability. All required modules are present and fully functional.

Module / File	Location	Responsibility
app.py	backend/	Flask app, routing, file upload handling
processing.py	backend/modules/	Core dataset analysis, chart generation
validation.py	backend/modules/	Regex-based user input validation
threading_tasks.py	backend/modules/	Multithreading wrapper
multiprocessing_tasks.py	backend/modules/	Multiprocessing wrapper
serialization.py	backend/modules/	JSON save/load of results

Module / File	Location	Responsibility
base_processor.py	backend/modules/	Abstract base class (ABC)
decorators.py	backend/utils/	log_function and timer decorators
generators.py	backend/utils/	data_generator and multiplier (closure)
iterators.py	backend/utils/	Custom DatasetIterator class
mixins.py	backend/utils/	LoggingMixin and ExportMixin
requirements.txt	root/	All project dependencies (Flask, Pandas, NumPy, Matplotlib, etc.)
README.md	root/	Setup instructions and project overview
summary.py	root/	Utility to collect all .py files into one text

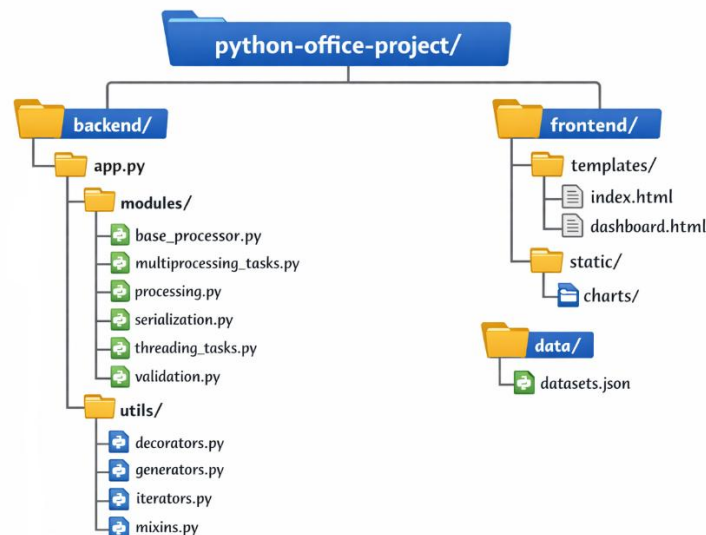


Fig. 1 — Project folder tree / directory structure

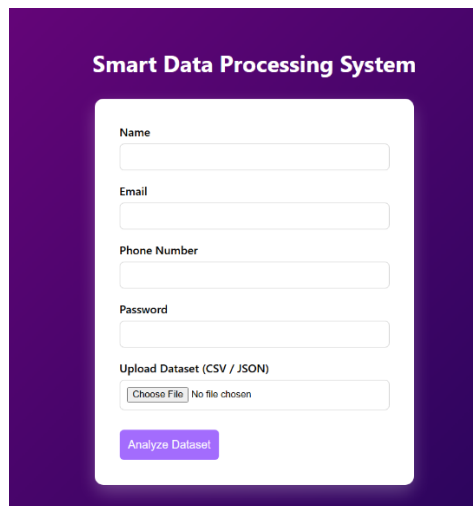
4. Module-wise Description

4.1 app.py — Flask Application

The main entry point of the web application. It initialises the Flask app with separate template and static folder paths pointing to the frontend directory. Three routes are defined:

- GET / — Renders the home (login/register) page.
- POST /submit — Handles user form submission, validates input, saves and processes the uploaded dataset, then renders the analytics dashboard.
- POST /update_column_chart — Accepts JSON with selected column axes and returns paths to freshly generated custom charts.

Multi-threading and multiprocessing are both triggered inside the /submit route to demonstrate concurrent execution alongside the main processing call.



Smart Data Processing System

Name

Email

Phone Number

Password

Upload Dataset (CSV / JSON)
 No file chosen

Fig. 2 — Home page (user registration form)

4.2 processing.py — Core Analytics Engine

This is the most feature-rich module. It contains the AdvancedProcessor class (which inherits from BaseProcessor, LoggingMixin, and ExportMixin) and the process_dataset() function decorated with @log_function and @timer. On receiving a valid CSV or JSON file path, it performs:

- Reads the dataset with Pandas; extracts numeric columns.
- Iterates through data using a custom generator (data_generator) and accumulates column sums and counts.
- Calculates mean, median, and standard deviation for each numeric column.
- Uses a closure (multiplier(2)) to compute doubled mean values.
- Generates three chart types per numeric column — line, bar, and histogram — saved as PNG files to frontend/static/charts/.
- Runs the custom DatasetIterator over the dataset rows.
- Calls the AdvancedProcessor to log and export results.
- Serialises the final statistics dictionary to JSON via serialization.py.

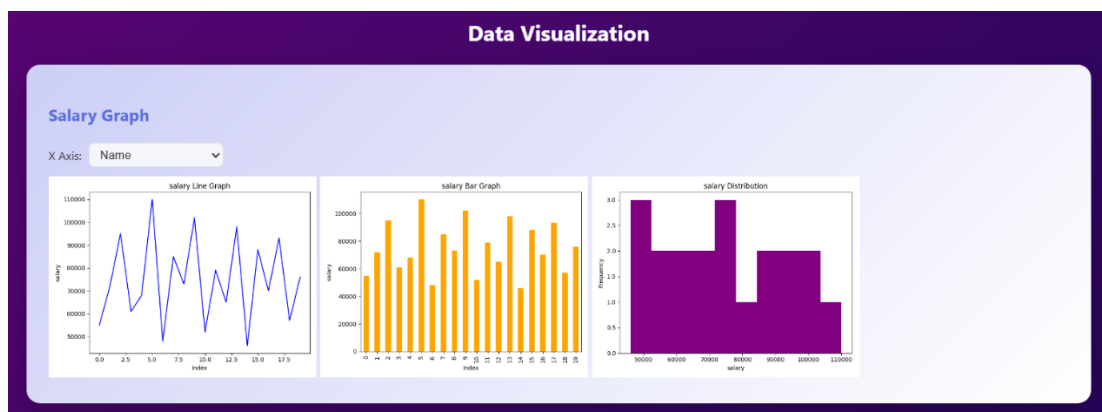


Fig. 3 — Analytics dashboard showing auto-generated charts

4.3 validation.py — Input Validation

Uses Python's re (regex) module to validate four form fields before any processing takes place. The patterns enforce:

Field	Pattern	Rule
Name	^[A-Za-z]+\$	Alphabets and spaces only
Email	^[w\.-]+@[w\.-]+\w+\$	Standard email format
Phone	^[0-9]{10}\$	Exactly 10 digits
Password	^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{6,}\$	Min 6 chars, letters + digits

4.4 Multithreading & Multiprocessing

threading_tasks.py wraps any callable in a Thread, while multiprocessing_tasks.py wraps it in a Process. Both are invoked within the /submit route to process the uploaded dataset concurrently, demonstrating Python's concurrent processing at a lower level. The sample_task in multiprocessing prints the chunk size, illustrating data partitioning.

4.5 Serialization — JSON Persistence

serialization.py provides a save_results() function that reads an existing datasets.json file (if present), merges the latest run statistics under the key latest_run, and writes it back. This gives a lightweight persistence mechanism without requiring a database.

4.6 Utilities: Decorators, Generators, Iterators, Mixins

Decorators: @log_function prints the function name before execution; @timer measures and prints elapsed time in seconds. Both are applied to process_dataset().

Generators: data_generator() yields rows one-by-one from a 2D list — memory efficient for large datasets. multiplier() is a closure factory that returns a configured multiply function.

Iterators: DatasetIterator implements __iter__, __next__, __len__, __str__, and __add__ — a full custom iterator demonstrating Python's data model including operator overloading via __add__.

Mixins: LoggingMixin provides a log() method; ExportMixin provides export_data(). AdvancedProcessor inherits from both alongside the ABC, demonstrating multiple inheritance.

5. OOP Concepts Demonstrated

Concept	Where Used	Description
Abstract Base Class	base_processor.py	BaseProcessor defines abstract process() method

Concept	Where Used	Description
Inheritance	processing.py	AdvancedProcessor inherits BaseProcessor, LoggingMixin, ExportMixin
Multiple Inheritance	processing.py	AdvancedProcessor has 3 parent classes
MRO (Method Resolution Order)	processing.py	Python resolves methods left-to-right: BaseProcessor → LoggingMixin → ExportMixin → object, ensuring no diamond-problem conflicts
Mixins	mixins.py	LoggingMixin and ExportMixin add reusable behaviours
Encapsulation	validation.py	All validation logic encapsulated in validate_user()
Operator Overloading	iterators.py	DatasetIterator implements __add__ to support + operator between iterator instances
Decorators (OOP)	decorators.py	Function wrappers using closures
Custom Iterator	iterators.py	DatasetIterator with full data model methods (__iter__, __next__, __len__, __str__, __add__)
Closure	generators.py	multiplier() returns an inner function capturing factor
Generator	generators.py	data_generator() uses yield for lazy evaluation

6. Technologies & Libraries Used

Library / Technology	Purpose
Flask	Web framework — routing, templating, request handling
Pandas	CSV/JSON reading, column extraction, HTML table rendering
NumPy	Numeric type detection, array operations
Matplotlib	Auto-generation of line, bar, and histogram charts
Python threading	Multithreaded background processing
Python multiprocessing	Multi-core parallel processing
re (regex)	Form input validation
json / os	Serialisation, file system operations
math / sys / datetime	Core library demonstration
HTML / CSS / Jinja2	Frontend templates for home and dashboard pages
venv (Virtual Environment)	Isolated Python environment for dependency management

7. Virtual Environment & Project Setup

The project uses Python's built-in venv module to create an isolated virtual environment, ensuring that all dependencies (Flask, Pandas, NumPy, Matplotlib, etc.) are contained and do not conflict with system-level packages.

Setup steps:

- `python -m venv venv`
- `venv\Scripts\activate` (Windows) or `source venv/bin/activate` (Mac/Linux)
- `pip install -r requirements.txt`
- `python backend/app.py`

All dependencies are listed in `requirements.txt` at the project root, making the project fully reproducible on any machine. A `README.md` file at the root provides complete setup and run instructions.

8. Application Flow

- User opens the application in a browser and fills in the registration form (name, email, phone, password) and optionally uploads a CSV/JSON file.
- On form submission, Flask's `/submit` route receives the POST request.
- `validate_user()` checks all four fields via regex; invalid inputs return an error immediately.
- If a file is attached, it is saved to the `data/` folder.
- `run_threading()` and `run_process()` execute `process_dataset()` and `sample_task()` concurrently.
- `process_dataset()` computes statistics, generates all charts, runs the iterator, invokes `AdvancedProcessor`, and serialises results.
- The dashboard is rendered with stats, a dataset preview table, and all generated charts.
- The user may select custom X/Y axes; the `/update_column_chart` endpoint regenerates and returns new chart paths via AJAX.

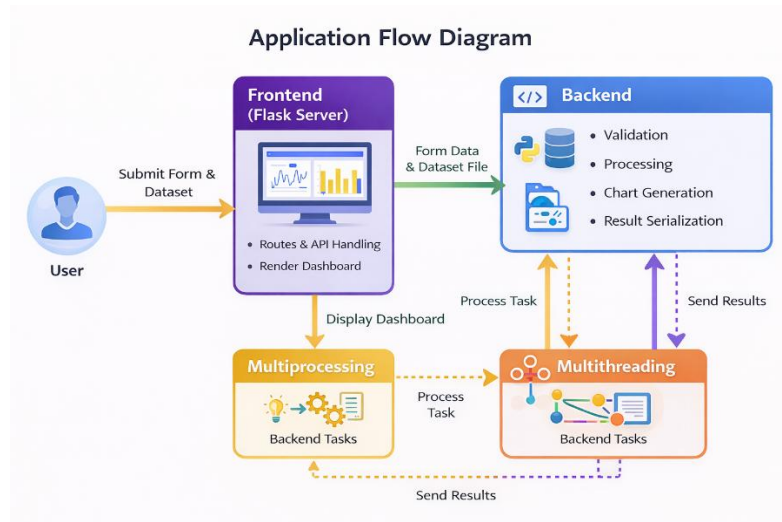


Fig. 4 — Application flow diagram

9. Key Code Snippets

9.1 Decorator Usage

```
@log_function
@timer
```

```
def process_dataset(filepath):  
    # Logs function name and measures execution time automatically
```

9.2 Generator & Closure

```
def data_generator(data):  
    for row in data:  
        yield row        # lazy row-by-row iteration
```

```
def multiplier(factor):  
    def inner(value):  
        return value * factor # closure captures factor  
    return inner
```

9.3 Abstract Class + Multiple Inheritance + MRO

```
class AdvancedProcessor(BaseProcessor, LoggingMixin, ExportMixin):  
    def process(self, data):  
        self.log('Processing dataset using AdvancedProcessor')  
        return self.export_data(data)
```

Python resolves method calls using the C3 linearisation algorithm (MRO). For `AdvancedProcessor`, the resolution order is: `AdvancedProcessor` → `BaseProcessor` → `LoggingMixin` → `ExportMixin` → object. This ensures that abstract methods from `BaseProcessor` are implemented before mixin methods are applied, with no ambiguity.

9.4 Custom Iterator with Operator Overloading

```
class DatasetIterator:  
    def __iter__(self): return self  
    def __next__(self):  
        if self.index >= len(self.data): raise StopIteration  
        value = self.data[self.index]; self.index += 1; return value  
    def __add__(self, other):        # operator overloading  
        return DatasetIterator(self.data + other.data)
```

10. Results & Output

When a valid CSV file is uploaded, the system produces the following outputs:

- A statistical summary table per numeric column (mean, median, std dev) rendered on the dashboard.
- Three charts per numeric column (line, bar, histogram) saved as PNG and displayed on the dashboard.
- A custom chart based on user-selected X/Y axes generated via AJAX without page reload.
- A JSON file (`data/datasets.json`) persisting the latest run statistics.
- Console logs showing function names, execution times, processing info, and iterator state.

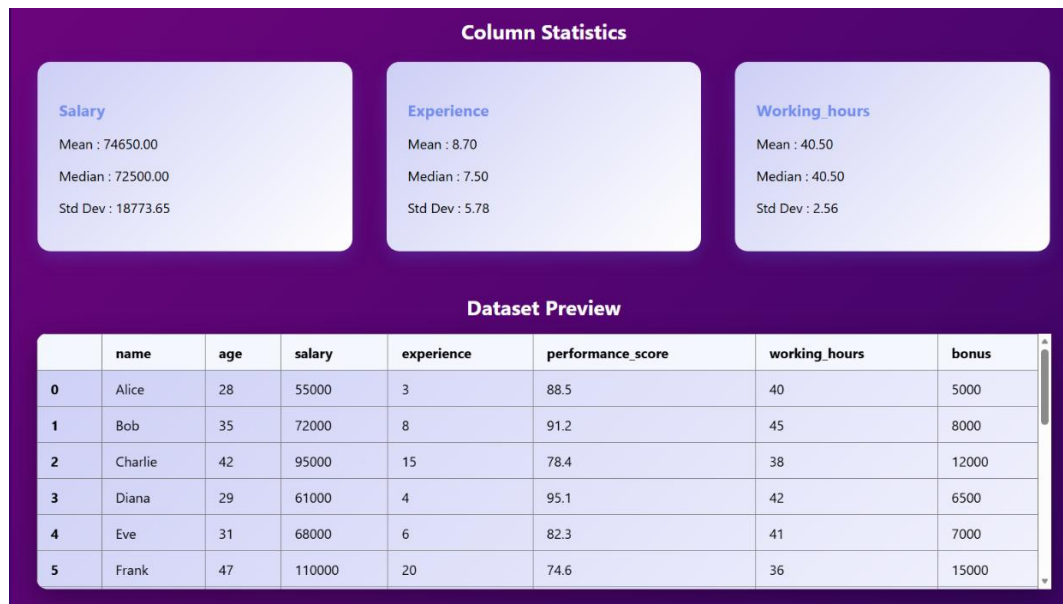


Fig. 5 — Dashboard output: statistics

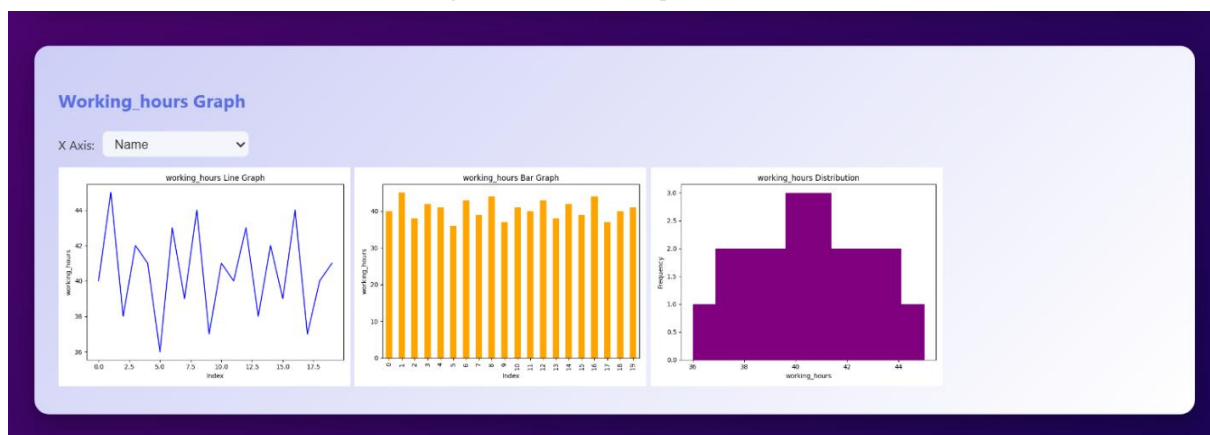


Fig. 6 — Auto-generated visualisations

11. Challenges & Solutions

Challenge	Solution Applied
Matplotlib saving charts in a headless server environment	Used matplotlib.use('Agg') backend before importing pyplot
Threading and multiprocessing sharing the same file path	Ensured absolute paths via os.path.abspath() before passing to threads
Clearing old charts before generating new ones	Listed and deleted all files in static/charts/ at start of each run
Windows path separator causing issues in HTML src attributes	Replaced backslashes with forward slashes using filepath.replace('\\', '/')
Keeping generator memory-efficient for large datasets	Used yield in data_generator instead of building a full list

12. Evaluation Criteria Alignment

The table below maps each grading criterion to where it is demonstrated in the project:

Criterion	Where Demonstrated
Use of Python Concepts	Decorators (@log_function, @timer), generators (data_generator), closures (multiplier), custom iterator (DatasetIterator with __add__), ABC (BaseProcessor), multiple inheritance + MRO (AdvancedProcessor), mixins (LoggingMixin, ExportMixin)
Backend Implementation	Flask app with 3 routes, form handling, AJAX endpoint, regex validation, threading & multiprocessing in /submit
Data Processing	Pandas CSV/JSON reading, NumPy numeric detection, mean/median/std dev per column, JSON serialisation
Visualization	Matplotlib line, bar, and histogram charts per column; user-selectable custom X/Y axis charts via AJAX
Code Structure	Modular layout: backend/modules/, backend/utils/, frontend/; separate responsibilities per file; requirements.txt; venv
Documentation	This report covers all concepts, challenges, code snippets, and application flow. README.md provides setup instructions.
Innovation	AJAX-based dynamic chart regeneration without page reload; full DatasetIterator data model; closure-powered stat multiplication

13. Conclusion

The Smart Data Processing & Validation Dashboard successfully demonstrates the integration of advanced Python programming paradigms within a real-world web application. By combining Flask routing, Pandas data analysis, Matplotlib visualisation, and concurrent processing, the project goes beyond basic scripting to showcase professional software engineering practices.

All required concepts from the assignment are practically applied: abstract base classes, multiple inheritance with MRO, operator overloading, mixins, decorators, closures, generators, custom iterators, regex validation, JSON persistence, threading, multiprocessing, and virtual environment management. The result is a functional, browser-accessible data analytics dashboard suitable for processing and visualising any CSV or JSON dataset.

This project provides a strong foundation that can be extended with database integration, user authentication, and more sophisticated machine learning analytics in future iterations.

14. References

- Flask Documentation — <https://flask.palletsprojects.com>
- Pandas Documentation — <https://pandas.pydata.org/docs>
- Matplotlib Documentation — <https://matplotlib.org/stable/contents.html>
- Python Official Docs (threading, multiprocessing, abc, venv) — <https://docs.python.org/3>
- NumPy Documentation — <https://numpy.org/doc>